StudySpot – Project Specification

2.1 Project Introduction

StudySpot is a mobile app that shows students the libraries, cafés and university buildings near them, in a searchable list and on a map, and keeps them available offline.

Team members	Mark Stessl, Nippon Lama
Working title	StudySpot
Course	Cross-Platform Mobile Development, WS 2026, Group O

What the app does. The app finds study places around the user's GPS position through an external places API. Users can search and filter them, see them on a map, open details, save favorites, and still use previously loaded data without internet.

Target users. University and college students, commuters and anyone in Vienna looking for a place to study.

Problem. Study places are spread across different services, and places a student found once are hard to find again. StudySpot combines nearby search, a map, favorites and offline access in one app.

2.2 Chosen Framework

StudySpot is built with React Native and TypeScript on Expo SDK 58. Android is the primary platform; the app was also run on iOS through Expo Go.

Why React Native. Both team members have hands-on React.js experience with components, props, state and hooks. That knowledge transfers directly to React Native, so the team could focus on mobile-specific features such as GPS, maps and offline storage. TypeScript catches data-model errors at compile time.

Target platforms. Android (emulator and device) is required and fully tested. iOS was tested on an iPhone with Expo Go.

Area	Technology used
Language and runtime	TypeScript, React Native 0.88, Expo SDK 58
Navigation	React Navigation: bottom tabs and native stacks

Area	Technology used
Shared state	Zustand with persist middleware
Server state and caching	TanStack Query
Local storage	AsyncStorage
Native feature	Expo Location (GPS)
Map	react-native-maps
Icons and layout	@expo/vector-icons, react-native-safe-area-context
Testing	Jest (jest-expo preset), React Native Testing Library

2.3 Planned Features (User Stories)

All nine user stories are implemented. Stories 1–8 come from the original specification with their acceptance criteria made concrete; story 9 was added because the Settings screen gained its own features (see 2.9).

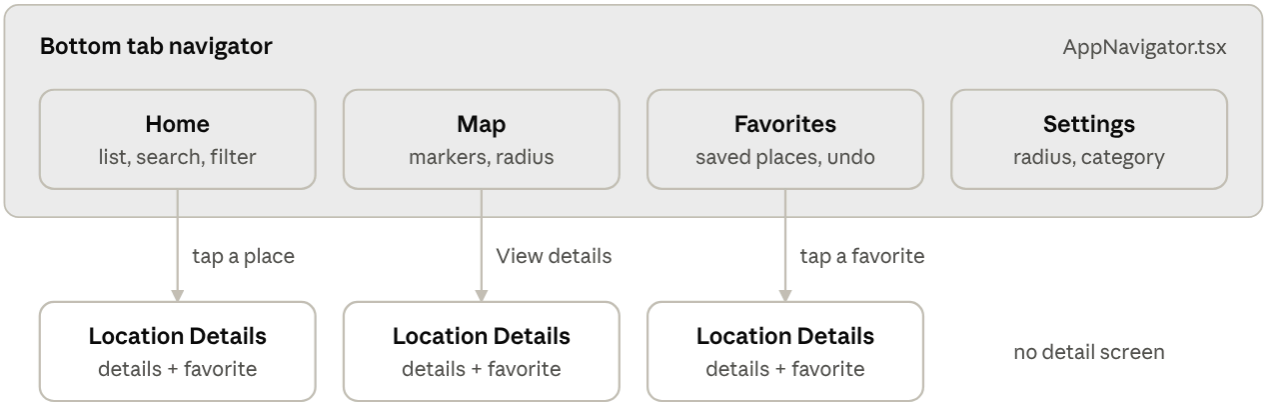
#	User story	Done when…
1 Nearby Study Locations	As a student, I want to see study locations near me, so that I can quickly find a suitable place to study.	The user location is requested; nearby places are loaded from the API within the search radius and sorted by distance; loading, error, empty and result states are shown.
2 Map View	As a student, I want to see study locations on a map, so that I understand where they are relative to me.	The map shows the user position (or the default location), the search radius and one marker per place; selecting a marker shows name, category, distance and address and opens the details.
3 Location Details	As a student, I want to view details about a study location, so that I can decide whether it is suitable.	Name, category, address, distance, coordinates, opening hours, website and OpenStreetMap ID are shown; the place can be added to or removed from favorites.

#	User story	Done when…
4 Favorites	As a student, I want to save and view favorite study locations, so that I can quickly find them again.	Favorites can be added and removed (with undo), are stored locally, survive app restarts and are listed on the Favorites screen; selecting one opens its details.
5 Search & Filter	As a student, I want to search and filter study locations, so that I can quickly find relevant places.	Places can be searched by name (ignoring case and accents) and filtered by Library, Cafe or University; search and filter can be reset with one tap.
6 Device Location	As a student, I want the app to use my current location, so that nearby places can be found and sorted by distance.	Location permission is requested at runtime; a granted permission provides coordinates for search and distances; denial or missing GPS is handled safely with the default location and a message.
7 Offline Usage	As a student, I want previously loaded locations to remain available without internet, so that I can keep using the app offline.	The last API result is cached locally; cached places and favorites are available offline; an offline banner shows the age of the cached data.
8 Refresh & Error Handling	As a user, I want to refresh data and receive understandable error messages, so that I know whether information is current.	Pull-to-refresh and a Refresh button start a new request; success updates the cache; network, timeout, server, API-key, invalid-response and empty-result errors show a clear message without crashing.
9 Personal Settings	As a student, I want to set my search radius, preferred category and location behaviour, so that results fit how I search.	Radius (1, 2, 5, 10 km), preferred category and device or default location can be chosen and persist across restarts; the cache can be deleted without losing favorites; settings can be reset.

2.4 Screen and Navigation Structure

The app has five screens: four bottom tabs (Home, Map, Favorites, Settings) and Location Details, which opens inside the Home, Map and Favorites tabs.

Home, Map and Favorites each open Location Details in their own stack



Each tab has its own stack (HomeStackNavigator.tsx), so Back returns to where the user came from.

navigation · 4 tabs, 3 detail stacks

React Navigation provides the tabs (`AppNavigator.tsx`) and one native stack per tab (`HomeStackNavigator.tsx`). The tab bar stays visible on Location Details.

Screen	What is shown	Internal state
Home	Location status, search bar, category chips, nearby list sorted by distance, "X of Y places" with Reset, pull-to-refresh, loading, error, empty and offline states	search text and category (`useState`); places, location and errors from `useLocations`
Map	Map with user position or default location, search-radius circle, one marker per place, info card of the selected place with View details, recenter button	selected place and map reference (`useState` , `useRef`); places from `useLocations` ; radius from `settingsStore`
Location Details	Category, name, distance, favorite button, address, coordinates, opening hours, website link, OpenStreetMap ID	place from the route parameter; favorite state from `favoritesStore` ; origin from `useCurrentLocation`
Favorites	Saved places sorted by distance, Remove action, undo bar, empty state	favorites from `favoritesStore` ; last removed place (`useState`)

Screen	What is shown	Internal state
Settings	Search radius, preferred category, location mode with permission status, cache info, Refresh, Delete cache, Reset settings	`settingsStore`; cache info from `useLocations`

2.5 API and Data Sources

Study places come from the Geoapify Places API, which serves OpenStreetMap data and needs a free API key but no user account. The original plan used the Overpass API; 2.9 explains the switch.

Request. One GET request to `https://api.geoapify.com/v2/places` with these parameters:

Parameter	Value
`categories`	`education.library`, `catering.cafe`, `education.university`, `education.college`
`filter`	`circle:<lon>,<lat>,<radius>` around the device or default location; radius from Settings (default 2 km)
`bias`	`proximity:<lon>,<lat>`
`limit`	200
`apiKey`	from `EXPO_PUBLIC_GEOAPIFY_API_KEY` in `.env` (not committed; `.env.example` documents it)

Data used. OpenStreetMap ID and type, name, coordinates, category (college is shown as University), address, website and opening hours. Places without a name or OpenStreetMap ID are skipped, and duplicates are removed. Because the IDs stay OpenStreetMap IDs (e.g. `node-123`), saved favorites stay valid.

Screen mapping. Home shows the data as a list, Map as markers, and Location Details shows one selected place.

Error handling. Every failure becomes a typed `ApiError` with a user-facing message; technical details such as HTTP codes are never shown.

Error	Trigger	Behaviour
network	no connection	one automatic retry; cached places stay visible with the offline banner
timeout	no answer within 15 s	message, one automatic retry
server	HTTP error from the API	message, one automatic retry
apiKey	key missing, or HTTP 401/403	message that the key is missing or invalid
invalid	response is not valid place data	message
empty	no places within the radius	message suggesting a larger radius

Fallback. If a request fails, the last cached result is shown. There is no mock data: without a cache and network the app shows the error with a Try again button.

2.6 Native Feature Usage

The native feature is GPS / device location through Expo Location. It decides where the app searches, how places are sorted and where the user appears on the map.

Where it is used.

- **Search origin:** the API request is centred on the device position.
- **Distance:** every place gets a straight-line (Haversine) distance; Home and Favorites are sorted by it, and Location Details shows it.
- **Map:** the user's position is shown as the native blue dot.

How it works.

1. Foreground permission is requested at runtime (`requestForegroundPermissionsAsync`). The Android permission text is set in `app.json`.
2. A fresh position is requested with high accuracy and a 12 s limit.
3. If no fix arrives (common indoors and on emulators), the last known position up to 30 minutes old is used.
4. If permission is denied or no position is available, the app keeps working around a predefined default location, FH Technikum Wien (48.239, 16.3776), and shows a short message.

User control. In Settings, users choose between *Use my location* and *Use default location*; the second never asks for permission. Settings shows the permission status, with a shortcut to the system settings after a denial. The app never tracks location in the background, and coordinates are only stored as the origin of the cached API result.

The native map view from react-native-maps supports this feature but is not counted as the required native capability.

2.7 Offline and Persistence Strategy

Favorites, settings and the last successful API result are stored on the device with AsyncStorage, so the core experience works without internet.

Area	Storage key	What is stored	Why
Favorites	`@studyspot/favorites/v2`	Saved places without distance (distance depends on the current position), via a persisted Zustand store	Saved places must survive restarts and work offline
API cache	`@studyspot/location-cache/v2`	Last successful places plus timestamp, search origin and radius	Places stay available offline; the timestamp drives the offline banner
Settings	`@studyspot/settings/v1`	Search radius, preferred category, location mode, via a persisted Zustand store	Preferences survive restarts

Synchronisation.

1. Start the app.
2. Load the local cache and show the cached places right away.
3. Get the current position, then request current data from the API.
4. On success, update the screen and overwrite the cache.
5. On failure, keep showing the cached data with an offline banner ("Offline · showing saved data from 5 min ago") and the error message.

Data counts as fresh for 5 minutes. After that, it is refetched when the app returns to the foreground, on pull-to-refresh, or when the radius or position changes.

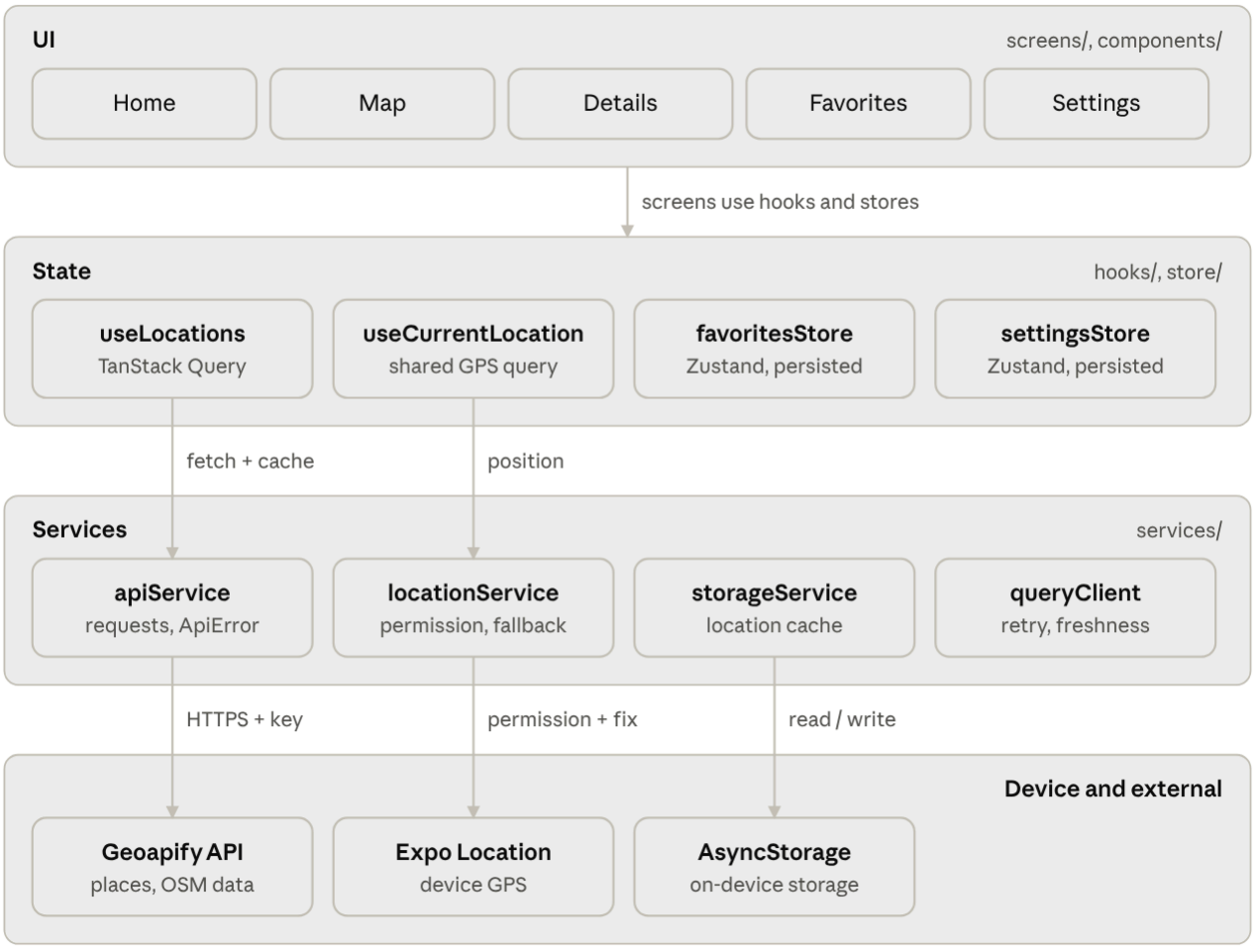
Offline scenario. Without internet, the app shows cached places and all favorites; distances, search, filters and details keep working. The map still shows the cached markers, but its background tiles may not load without internet. New places can be loaded once the connection returns.

Cache deletion. Settings can delete the API cache after a confirmation. Favorites and settings are kept.

2.8 Technical Approach and Architecture

The app uses a layered Screens + Hooks architecture: screens only render and handle input, hooks and stores hold state, and services talk to the API, GPS and storage.

Screens never call the API directly: hooks and services sit between them



architecture · 4 layers

The two stores persist to AsyncStorage through Zustand's persist middleware; `useLocations` also reads and writes the cache through `storageService`.

Folder structure (`src/`).

Folder	Content
`components/`	`LocationCard`, `SearchBar`, `CategoryFilter`, `LoadingView`, `ErrorView`, `OfflineBanner`, `InfoBanner`, `ScreenHeader`, `ScreenContainer`, `AppErrorBoundary`
`screens/`	`HomeScreen`, `MapScreen`, `LocationDetailsScreen`, `FavoritesScreen`, `SettingsScreen`
`navigation/`	`AppNavigator` (tabs), `HomeStackNavigator` (detail stacks), `navigationTypes`, `useOpenLocationDetails`
`services/`	`apiService`, `locationService`, `storageService`, `queryClient`
`store/`	`favoritesStore`, `settingsStore`
`hooks/`	`useLocations`, `useCurrentLocation`
`types/`	`Location`, `Settings`
`utils/`	`distance`, `filters`
`constants/`	`theme`, `storageKeys`, `studySpot` (default location, API settings, radius options)

State. React `useState` holds temporary UI state such as search text and the selected marker. Zustand holds favorites and settings. TanStack Query handles API requests, caching, loading and error states and refetching; the current position is also a shared query, so the permission prompt appears only once.

Testing. Jest (jest-expo preset) and React Native Testing Library run 41 tests in 8 suites: distance and filter utilities, Geoapify mapping and error handling, the location fallback, the persisted favorites store, and the `LocationCard`, `CategoryFilter` and `OfflineBanner` components. `npm run typecheck` checks all TypeScript. Screens were tested manually on the Android emulator and on an iPhone with Expo Go.

2.9 Changelog

The final build keeps the original concept; the main change is the data source, which moved from the Overpass API to Geoapify.

Date	Change	Reason
Nov 2026	Tests moved from Node's test runner to Jest and React Native Testing Library; unit and component tests added (41 tests)	Match the planned testing stack and allow component tests
Nov 2026	Settings gained search radius, preferred category and location mode; user story 9 (Personal Settings) added	Settings planned in 2.4 needed their own story and acceptance criteria
Nov 2026	Location Details completed with a favorite action; Favorites got a Remove action with undo	Meet the acceptance criteria of stories 3 and 4; undo prevents accidental removals
Nov 2026	Home rebuilt with location status, search, category filter and reset; filter logic moved to `utils/filters.ts`	Meet story 5 and keep business logic out of screens
Nov 2026	Map screen implemented with react-native-maps (it was a placeholder before)	Meet story 2
Nov 2026	Navigation restructured into Home, Map, Favorites and Settings tabs with nested detail stacks; the separate Nearby tab was merged into Home	Match the planned navigation structure
Nov 2026	Places API changed from the public Overpass API to the Geoapify Places API (still OpenStreetMap data, free API key); requests moved to TanStack Query with a timestamped offline cache	The public Overpass server often timed out or was overloaded, so places did not load reliably
Nov 2026	Location lookup waits up to 12 s for a fix and falls back to the last known position	The Android emulator and indoor devices often delivered no fresh fix, which showed "location unavailable" despite GPS being on
Nov 2026	Built-in sample data removed; categories limited to Library, Cafe and University (Coworking removed)	Sample places looked like real results; align categories with the specification

Date	Change	Reason
Nov 2026	Favorites and settings moved from a React Context to persisted Zustand stores	Use the state management planned in 2.8
Nov 2026	Minimal UI redesign with shared components (SearchBar, CategoryFilter, LoadingView, ErrorView, OfflineBanner)	Consistent, cleaner UI and the components planned in 2.8
Nov 2026	App-level error boundary added	An unexpected rendering error shows a recovery screen instead of a blank app
Nov 2026	First full implementation: Expo SDK 58, Overpass API integration, GPS and AsyncStorage services, shared state in a React Context, Explore, Nearby, Saved and Settings screens, unit tests for distances and API mapping	Build a working version early, then align it with the planned libraries and structure
Sep 2026	Repository created from the course template; first app architecture, navigation tabs, home screen and reusable location card	Start of implementation
Initial submission	First StudySpot specification	Project planning

Traceability: User Stories to Code

Each user story maps to the files that implement it and the tests that cover it. All paths are relative to `src/`.

#	User story	Implemented in	Tested in
1	Nearby Study Locations	`screens/HomeScreen.tsx`, `hooks/useLocations.ts`, `services/apiService.ts`, `utils/distance.ts`, `components/LocationCard.tsx`, `components/LoadingView.tsx`, `components/ErrorView.tsx`	`services/apiService.test.ts`, `utils/distance.test.ts`, `components/__tests__/LocationCard.test.tsx`
2	Map View	`screens/MapScreen.tsx`, `hooks/useLocations.ts`, `navigation/useOpenLocationDetails.ts`	manual (Android emulator, iPhone)

#	User story	Implemented in	Tested in
3	Location Details	`screens/LocationDetailsScreen.tsx`, `store/favoritesStore.ts`, `utils/distance.ts`, `navigation/HomeStackNavigator.tsx`	`store/favoritesStore.test.ts`, `utils/distance.test.ts`
4	Favorites	`screens/FavoritesScreen.tsx`, `store/favoritesStore.ts`, `components/LocationCard.tsx`	`store/favoritesStore.test.ts`, `components/__tests__/LocationCard.test.tsx`
5	Search & Filter	`screens/HomeScreen.tsx`, `utils/filters.ts`, `components/SearchBar.tsx`, `components/CategoryFilter.tsx`	`utils/filters.test.ts`, `components/__tests__/CategoryFilter.test.tsx`
6	Device Location	`services/locationService.ts`, `hooks/useCurrentLocation.ts`, `utils/distance.ts`, `app.json` (project root, permission text)	`services/locationService.test.ts`
7	Offline Usage	`services/storageService.ts`, `hooks/useLocations.ts`, `components/OfflineBanner.tsx`, `store/favoritesStore.ts`	`components/__tests__/OfflineBanner.test.tsx`, `store/favoritesStore.test.ts`
8	Refresh & Error Handling	`services/apiService.ts` (`ApiError`), `services/queryClient.ts`, `screens/HomeScreen.tsx` (pull-to-refresh), `components/ErrorView.tsx`, `components/AppErrorBoundary.tsx`	`services/apiService.test.ts`
9	Personal Settings	`screens/SettingsScreen.tsx`, `store/settingsStore.ts`, `types/Settings.ts`, `constants/studySpot.ts`, `hooks/useLocations.ts` (cache deletion)	manual (Android emulator)

Navigation for all stories is defined in `navigation/AppNavigator.tsx` (tabs) and `navigation/HomeStackNavigator.tsx` (detail stacks).